

# ELI — Portable Scripts Guide

ELI v2.1.15

July 2026

## Contents

|                                                   |          |
|---------------------------------------------------|----------|
| <b>Which script do I run? — ELI v2.0 portable</b> | <b>1</b> |
| The short answer . . . . .                        | 1        |
| Pick your path (decision table) . . . . .         | 2        |
| Important: ELI needs an AI model . . . . .        | 2        |
| Every file, explained . . . . .                   | 2        |
| After it's running — everyday use . . . . .       | 4        |
| If something goes wrong . . . . .                 | 4        |

## Which script do I run? — ELI v2.0 portable

**Updated for v2.1.15 (July 2026).** The primary way to install ELI is now the **prebuilt installers on GitHub Releases** — `ELI-Setup-<v>.exe` (Windows), the `.dmg` (macOS, Apple Silicon), the `.AppImage` (Linux). The Linux AppImage and Windows installer are built and launch-tested in CI; the macOS `.dmg` is built on a Mac and provided best-effort (not verified in CI). First launch offers GPU acceleration (NVIDIA CUDA / AMD Vulkan; Apple Metal is built in) and a starter model sized to your hardware. Data lives in a per-user `ELI_v2` folder and survives upgrades; `--fresh-start` resets it. Everything below remains valid for **source installs** (clone + `install.sh`) and the classic portable tarball.

You have just extracted `ELI_v2-2.1.15-linux-portable` and you are looking at a folder full of scripts. This guide explains **every launcher in plain language**: what it is, when to use it, the exact command to type, and what it actually does under the hood. No prior knowledge assumed.

---

## The short answer

**After a fresh extract, open a terminal in this folder and run:**

```
chmod +x ELI_Setup.sh
./ELI_Setup.sh
```

That one command installs everything, sets ELI up, and launches it. If you only ever read one line of this document, that is the line.

`chmod +x` means “mark this file as runnable” — you only need it once. The `./` in front means “run the script that is right here in this folder.”

Everything else in this folder is either **(a)** something `ELI_Setup.sh` calls for you, **(b)** an alternative for people who want more control, or **(c)** a tool for developers that you should ignore.

---

## Pick your path (decision table)

| Your situation                                     | Run this                                                                      |
|----------------------------------------------------|-------------------------------------------------------------------------------|
| “Just make it work.”                               | <code>./ELI_Setup.sh</code>                                                   |
| I want to install now and launch later, separately | <code>./INSTALL_ELI.sh</code> then <code>./RUN_ELI.sh</code>                  |
| My computer has <b>no NVIDIA graphics card</b>     | <code>./INSTALL_ELI.sh --cpu-only</code> then <code>./RUN_ELI.sh</code>       |
| I want ELI’s AI model + voices too (big download)  | <code>./RUN_ELI.sh --with-github-assets</code>                                |
| I want to use ELI from my <b>phone or tablet</b>   | <code>./scripts/eli_serve.sh --lan --https</code>                             |
| I’m on <b>Windows</b>                              | <code>install.bat</code> (or the <code>Setup.exe</code> from GitHub Releases) |
| I’m a developer building release packages          | <code>build_packages.sh</code> — <i>not for normal use</i>                    |

---

## Important: ELI needs an AI model

A fresh portable ships **without** the large AI model file (to keep the download small). The `models/` folder starts empty. ELI will not think until a model is present. Two ways to get one:

- **Bundled convenience pack** (model + voices), pulled from the project’s GitHub Release:

```
gh auth login                # one-time: log in to GitHub
./RUN_ELI.sh --with-github-assets # downloads the model/voice pack, then launches
```

- **Automatic model download** (guided), if you prefer just the model:

```
.venv/bin/python -m eli.core.model_download --auto
```

`ELI_Setup.sh` offers to do the asset step for you, so most people never type these by hand.

---

## Every file, explained

### `ELI_Setup.sh` — the recommended one-click

**What it is:** the friendly “grandparent setup.” A double-click-friendly name that hands off to `scripts/eli_setup.sh`.

**When to use:** first time, and whenever you want the easy path. **Command:** `./ELI_Setup.sh` **What it does:** runs an 8-step first-run sequence — creates the private Python environment (`.venv`), installs all dependencies, sets up the databases, optionally fetches the model/voice pack, installs the `eli` terminal command and a desktop icon, opens a small graphical wizard (using pop-up dialogs if your system has them), and finally launches ELI. **To you:** the “do everything for me” button.

### `INSTALL_ELI.sh` — install only (no launch)

**What it is:** a thin wrapper around `scripts/eli_one_click_setup.sh`. **When to use:** you want to set ELI up now and start it yourself later. **Command:** `./INSTALL_ELI.sh` (add `--cpu-only` if you have no NVIDIA GPU) **What it does:** builds the `.venv`, installs dependencies (GPU/CUDA build by default), initialises the databases, and installs the `eli` command + desktop launcher. It does **not** open ELI — that’s what `RUN_ELI.sh` is for. **Useful options:** `--cpu-only` (no graphics card), `--skip-torch` (don’t install the heavy PyTorch library), `--with-github-assets` (also grab the model/voice pack). **To you:** “get it ready, but don’t open it yet.”

### `RUN_ELI.sh` — launch ELI (your daily button)

**What it is:** a wrapper around `scripts/eli_startup.sh`. **When to use:** every day, to start ELI. Safe to run even if you haven’t installed yet — it will install first if needed. **Command:** `./RUN_ELI.sh` **What it does:** checks that ELI is installed (installs if not), optionally restores the model/voice assets, then launches the app.

**Useful options:** `--with-github-assets` (fetch the model/voice pack before launching), `--safe-mode` (turn off proactive/experimental startup features if something misbehaves), `--trace` (verbose logs for troubleshooting), `--no-setup` (skip the install check). **To you:** the “open ELI” button.

### `install.sh` — the full Linux/macOS installer (advanced)

**What it is:** the low-level installer that the one-click scripts wrap. This is the real engine that builds the environment. **When to use:** you want direct control over the installation, or you’re comfortable in a terminal. **Command:** `bash install.sh` **What it does:** creates the `.venv`, installs the **exact, frozen** set of tested dependencies (reproducible), verifies GPU offload works, and initialises data folders and databases. It does **not** set up the friendly `eli` command, the wizard, or launch ELI — the one-click scripts add those niceties on top. **Useful options:** `--cpu-only` (no GPU), `--skip-torch` (skip PyTorch), `--latest` (newest dependency versions instead of the frozen lock), `--install-cuda` (best-effort install of the NVIDIA CUDA toolkit). **To you:** “the manual installer” — most people should use `ELI_Setup.sh` instead.

### `install.bat` and `install.ps1` — Windows installers

**What they are:** the Windows equivalent of `install.sh`. `install.bat` is the double-click file; it simply launches the more capable `install.ps1` (PowerShell). **When to use:** you’re on Windows. **Command (in the folder):** `install.bat` — or `install.bat /cpu` for no GPU. **What they do:** same idea as `install.sh` — build the environment, verify the GPU, initialise databases. **Useful options:** `/cpu` (CPU-only), `/cuda` (also install the CUDA toolkit via winget), `/latest` (newest versions instead of the frozen lock). **To you:** “the Windows installer.” Non-technical Windows users may prefer the `Setup.exe` from the GitHub Releases page instead.

### `eli.sh` — the bare launcher (low-level)

**What it is:** the minimal launcher. It points the environment variables at this folder and runs `python -m eli` inside the `.venv`. **When to use:** rarely — only if you’ve already installed and want the most direct start. **Command:** `./eli.sh` **What it does:** exactly one thing — starts ELI from the already-built `.venv`. If the environment isn’t there yet, it tells you to run the installer first. `RUN_ELI.sh` is the friendlier version of this. **To you:** the “engine start” with none of the conveniences.

### `eli` — the terminal command (created during setup)

**What it is:** a shortcut installed into `~/.local/bin/eli` by the setup scripts. **When to use:** after installing, from **any** folder. **Command:** `eli` **What it does:** launches ELI without needing to be inside this folder. **Tip:** if typing `eli` runs an old version, run `hash -r` once to refresh your shell.

### `eli-v2.0.desktop` — the desktop / menu shortcut

**What it is:** a Linux application-menu entry (the clickable icon). **When to use:** you don’t run it directly — the setup installs it so ELI appears in your applications menu with its icon, launchable by mouse. **What it contains:** the app name, icon, and the command to run (`scripts/eli_launch.sh gui`). The `__REPO_ROOT__` placeholders inside are filled in with this folder’s real path when the launcher is installed (via `./packaging/desktop/install_desktop_launcher.sh`). **To you:** the “ELI icon” in your menu once setup has run.

### `build_packages.sh` — developer tool (not for end users)

**What it is:** the script that **builds** the distributable packages (Python wheel, Linux AppImage, Windows portable zip, macOS bundle, etc.). **When to use:** only if you are packaging ELI for release. **What it does:** assembles installable artifacts under `dist/`. It does **not** run or install ELI for use. **To you:** ignore this one — it’s for whoever ships ELI, not for using it.

`scripts/eli_serve.sh` — use ELI from your phone / tablet

**What it is:** starts ELI's built-in web server so a browser on another device can reach it. **Command:** `./scripts/eli_serve.sh --lan --https` **What it does:** binds to your local network (`--lan`) with an access token, and enables HTTPS (`--https`) which browsers require before they'll allow the **microphone**. It prints a URL (and a QR code is available in the app's *Connect a phone* tab) to open on the phone. Everything still runs on **this** machine — nothing goes to any cloud. **To you:** the “talk to ELI from my couch on my phone” button.

`README_INSTALL.txt`

**What it is:** the short plain-text quick-start that ships in the folder. This PDF is the expanded, friendly version of it.

---

## After it's running — everyday use

- **Start ELI:** `./RUN_ELI.sh` (or just type `eli`, or click the menu icon)
- **Phone access:** `./scripts/eli_serve.sh --lan --https`
- **Your data lives here** (created automatically on first run): `artifacts/db/` (memories & conversations), `artifacts/runtime/`, and `config/`. Because this is a *portable* copy, everything stays inside this folder — you can move or back up the whole `ELI_v2-2.1.15-linux-portable` folder and nothing is lost.

## If something goes wrong

- **“Virtual environment not found”:** you launched before installing — run `./ELI_Setup.sh` (or `./INSTALL_ELI.sh`) first.
- **No NVIDIA GPU / install fails on GPU bits:** reinstall with `./INSTALL_ELI.sh --cpu-only`.
- **ELI has no model / won't answer:** fetch one with `./RUN_ELI.sh --with-github-assets` (after gh auth login) or `.venv/bin/python -m eli.core.model_download --auto`.
- **Odd behaviour at startup:** try `./RUN_ELI.sh --safe-mode`, or `--trace` for detailed logs written under `artifacts/startup/logs/`.

---

*ELI runs entirely on your own hardware. It is offline by default — turn on the **Net** toggle in the app only when you want it to reach the internet.*